

Model Compiler Suite for Aries

Developers Guide

version 0.9.0.2

January 16, 2025

Important Notice

MOBILINT, Inc. reserves the right to make changes to the information in this publication at any time without prior notice. All information provided is for reference purpose only. MOBILINT assumes no responsibility for possible errors or omissions, or for any consequences resulting from the use of the information contained herein.

This publication on its own does not convey any license, either express or implied, relating to any MOBILINT and/or third-party products, under the intellectual property rights of MOBILINT and/or any third parties.

MOBILINT makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does MOBILINT assume any liability arising out of the application or use of any product or circuit and specifically disclaims any and all liability, including without limitation any consequential or incidental damages.

Customers are responsible for their own products and applications. "Typical" parameters can and do vary in different applications. All operating parameters, including "Typicals" must be validated for each customer application by the customer's technical experts.

MOBILINT products are not designed, intended, or authorized for use in applications intended to support or sustain life, or for any other application in which the failure of the MOBILINT product could reasonably be expected to create a situation where personal injury or death may occur.

Customers acknowledge and agree that they are solely responsible to meet all other legal and regulatory requirements regarding their applications using MOBILINT products notwithstanding any information provided in this publication. Customer shall indemnify and hold MOBILINT and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, expenses, and reasonable attorney fees arising out of, either directly or indirectly, any claim (including but not limited to personal injury or death) that may be associated with such unintended, unauthorized and/or illegal use.

WARNING No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electric or mechanical, by photocopying, recording, or otherwise, without the prior written consent of MOBILINT. This publication is intended for use by designated recipients only. This publication contains confidential information (including trade secrets) of MOBILINT protected by Competition Law, Trade Secrets Protection Act and other related laws, and therefore may not be, in part or in whole, directly or indirectly publicized, distributed, photocopied or used (including in a posting on the Internet where unspecified access is possible) by any unauthorized third party. MOBILINT reserves its right to take any and all measures both in equity and law available to it and claim full damages against any party that misappropriates MOBILINT's trade secrets and/or confidential information.

Document Revision History

Doc Revision Number	Date	Description
0.9.0.2	January 16, 2025	Revised for v0.9.0.2
0.9.0.1	December 13, 2024	Revised for v0.9.0.1
0.9.0.0	November 27, 2024	Revised for v0.9.0.0
0.8.5	June 20, 2024	Revised for v0.8.5
0.8.4	May 20, 2024	Revised for v0.8.4
0.8.3	March 7, 2024	Revised for v0.8.3
0.8.2	February 23, 2024	Revised for v0.8.2
0.8.1	December 8, 2023	Revised for v0.8.1
0.8.0	November 2, 2023	Revised for v0.8.0
0.7.12	September 12, 2023	Revised for v0.7.12
0.7.11	August 31, 2023	Revised for v0.7.11
0.7.10	August 11, 2023	Revised for v0.7.10
0.7.9	August 11, 2023	Revised for v0.7.9
0.7.8	August 8, 2023	Revised for v0.7.8
0.7.7	June 30, 2023	Revised for v0.7.7
0.7	March 23, 2023	Revised for v0.7
0.6	August 10, 2022	Revised for v0.6
0.5	July 1, 2022	Revised for v0.5
0.4	February 23, 2022	Revised for v0.4
0.3	February 5, 2022	Revised for v0.3
0.2	December 1, 2021	Revised for v0.2

Table of Contents

1. INTRODUCTION	8
2. CHANGELOG	9
2.1 qubee v0.9.0.2 (January 2025)	9
2.2 qubee v0.9.0.1 (December 2024)	9
2.3 qubee v0.9.0.0 (November 2024)	9
2.4 qubee v0.8.5 (June 2024)	9
2.5 qubee v0.8.4 (May 2024)	9
2.6 qubee v0.8.3 (March 2024)	10
2.7 qubee v0.8.2 (February 2024)	10
2.8 qubee v0.8.1 (December 2023)	10
2.9 qubee v0.8.0 (November 2023)	10
2.10 qubee v0.7.12 (September 2023)	10
2.11 qubee v0.7.11 (August 2023)	10
2.12 qubee v0.7.10 (August 2023)	10
2.13 qubee v0.7.9 (August 2023)	10
2.14 qubee v0.7.8 (August 2023)	10
2.15 qubee v0.7.7 (June 2023)	10
2.16 qubee v0.7 (March 2023)	10
2.17 qubee v0.6 (August 2022)	11
2.18 qubee v0.5 (July 2022)	11
2.19 qubee v0.4 (February 2022)	11
2.20 qubee v0.3 (February 2022)	11
2.21 qubee v0.2 (December 2021)	11
3. INSTALLATION	12
3.1 System requirements	12
3.1.1 Reference System	12
3.1.2 Requirements and Recommended Packages	12
3.2 SDK Installation	12
3.2.1 Building Docker Container	12
3.2.2 Installation of qubee	13
4. TUTORIALS	14
4.1 Preparing Calibration Data	14
4.1.1 Pre-process raw calibration dataset and save it as numpy tensors	14
4.1.2 Use a pre-processing configuration YAML file	15
4.1.3 Use a manually defined pre-processing function	15
4.2 Compiling ONNX Models	16
4.3 Compiling PyTorch Models	17
4.4 Compiling TensorFlow/Keras Models	17
4.5 Compiling TensorFlow Lite Models	18
5. CPU OFFLOADING (BETA VERSION)	19
6. SUPPORTED FRAMEWORKS	20
6.1 Supported Operations (ONNX)	21
6.2 Supported operations (PyTorch)	24
6.3 Supported operations (TensorFlow/Keras/TensorFlow Lite)	27

7. API REFERENCE	30
7.1 Function: mxq_compile	30
7.1.1 Tips for choosing quantization methods and selecting arguments	34
7.1.2 Compiling Models with Custom Input (ONNX/Torch/TensorFlow)	35
7.2 Class: Model_Dict	35
7.2.1 Methods	37
7.2.2 Method Details	38
7.3 Function: make_calib	42
7.4 Fuction: make_calib_man	43
7.5 Pre-processing Configurations	44
7.5.1 Pre-processing Parameters	46
8. OPEN SOURCE LICENSE NOTICE	48
9. COPYRIGHT	49

List of Tables

Table	Title	Page
Table 6-1.	ONNX Supported Operations	22
Table 6-2.	PyTorch Supported Operations	25
Table 6-3.	TensorFlow Supported Operations	28
Table 7-1.	mxq_compile.....	31
Table 7-2.	Model_Dict Class	36
Table 7-3.	Model_Dict Methods.....	37
Table 7-4.	Model_Dict.compile.....	39
Table 7-5.	make_calib.....	43
Table 7-6.	make_calib_man.....	44
Table 7-7.	Pre-processing function API	45
Table 7-8.	GetImage.....	46
Table 7-9.	Pad 46	
Table 7-10.	Normalize.....	46
Table 7-11.	ResizeTorch.....	46
Table 7-12.	Resize.....	47
Table 7-13.	CenterCrop	47
Table 7-14.	SetOrder	47

List of Figures

Figure	Title	Page
Figure 1-1.	SDK Components	8
Figure 5-1.	SDK CPU Offloading	19
Figure 6-1.	Supported deep-learning frameworks	20

Confidential

1. Introduction

Mobilint® Model Compiler (i.e., Compiler) is a tool that converts models from deep learning frameworks (ONNX, PyTorch, Keras, TensorFlow, etc...) into Mobilint® Model eXeCUtable (i.e., MXQ), a format executable by Mobilint® Neural Processing Unit (NPU). This is the manual for the **qubee**, Mobilint's SDK. In this manual, you can learn how to use it, what frameworks it supports, etc. A set of functions you can use to interact with the SDK will be given below. Inputs to qubee is a trained deep learning model, its input shape, and calibration data. It will return MXQ (compiled model) as an output.

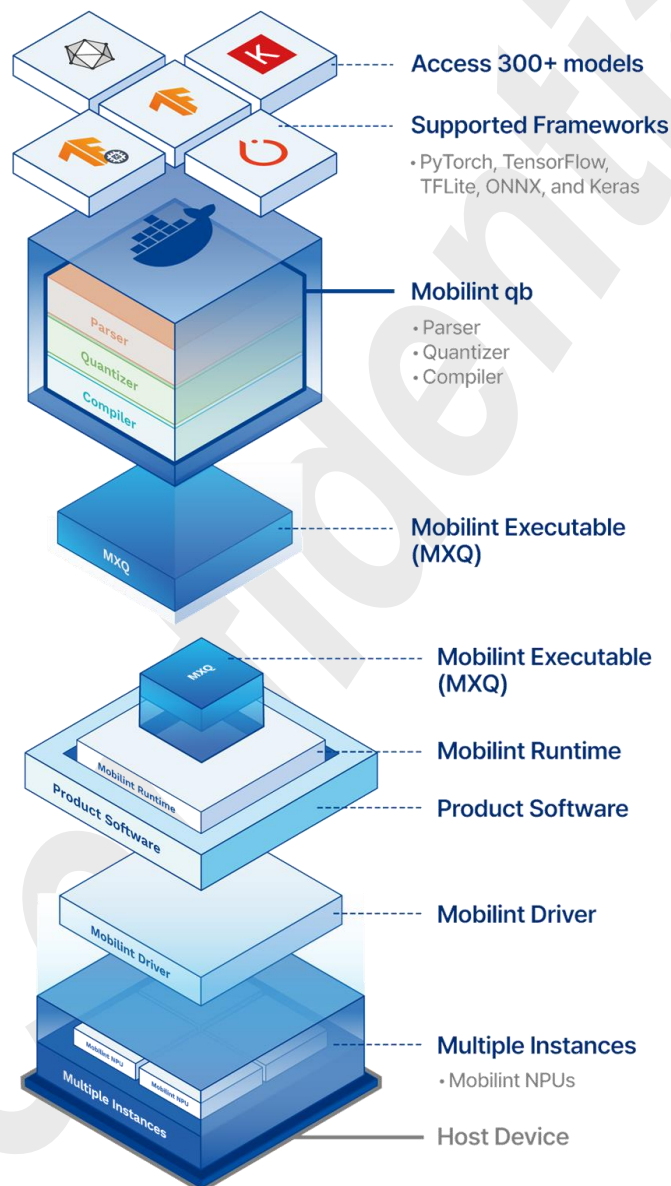


Figure 1-1. SDK Components

2. Changelog

2.1 qubee v0.9.0.2 (January 2025)

API

Add API for saving model architecture

Support Visualization tool

Support Run min output difference QAT mode

2.2 qubee v0.9.0.1 (December 2024)

API

Add API for global core (beta)

Add API for yolo post (beta)

Support diverse hardware (aries, regulus, aries2)

2.3 qubee v0.9.0.0 (November 2024)

API

Update the high-level parsing processes

Docker

onnx: 1.13.0 -> 1.16.2

tensorflow: 2.9.0 -> 2.17.0

torch: 1.13.0 -> 2.4.1

2.4 qubee v0.8.5 (June 2024)

API

Support FastPercentile quantization method

2.5 qubee v0.8.4 (May 2024)

API

TF backend connected to ONNX backend by TF2ONNX

Enable to compile models with custom input shape

Support more operations

Docker

onnx: 1.12.0 -> 1.13.0

2.6 qubee v0.8.3 (March 2024)

API

Support TF Lite backend

2.7 qubee v0.8.2 (February 2024)

2.8 qubee v0.8.1 (December 2023)

2.9 qubee v0.8.0 (November 2023)

API

TVM backend deprecated

2.10 qubee v0.7.12 (September 2023)

2.11 qubee v0.7.11 (August 2023)

API

Support TorchScript backend

2.12 qubee v0.7.10 (August 2023)

2.13 qubee v0.7.9 (August 2023)

2.14 qubee v0.7.8 (August 2023)

2.15 qubee v0.7.7 (June 2023)

API

Improve CPU offloading (beta version)

Improve CPU efficiency

Support more operations

Docker

torch: 1.10.1 -> 1.13.0

tensorflow: 2.3.0 -> 2.9.0

onnx:1.11.0 -> 1.12.0

2.16 qubee v0.7 (March 2023)

Multi-channel quantization

Support more operations

API

Improve calibration dataset processing

Support CPU offloading (beta version)

2.17 qubee v0.6 (August 2022)

Minor updates

2.18 qubee v0.5 (July 2022)

Docker

Conda -> Virtualenv

Python: 3.7.7 -> 3.8.10

torch: 1.8.1 -> 1.10.1

tensorflow: 1.15.0 -> 2.3.0

onnx:1.6.0 -> 1.11.0

Parser

Code refactoring

API

Enable saving sample inference results (inputs and outputs)

2.19 qubee v0.4 (February 2022)

Optimizer

Minor updates in fusing reshape

2.20 qubee v0.3 (February 2022)

Parser

Identify preprocess and postprocess of the model

Exclude preprocess and postprocess if they are unsupported by the NPU

API

Simulate integer inference in Python API

2.21 qubee v0.2 (December 2021)

First release

3. Installation

3.1 System requirements

We recommend to use NVIDIA GPU for faster compile with qubee, but it is not necessary. Currently, CPU version qubee is also supported.

3.1.1 Reference System

Ubuntu 22.04.4 LTS
NVIDIA Graphics Driver 545.29.06

3.1.2 Requirements and Recommended Packages

Ubuntu 20.04.6 LTS or Above
NVIDIA Graphics Driver 535.183.01 or Above
Docker
nvidia-docker

3.2 SDK Installation

We recommend installing qubee on the Mobilint docker container.

(Docker image: mobilint/qbcompiler:v0.9.0.2, <https://hub.docker.com/r/mobilint/qbcompiler>)

We periodically upload the latest Docker image to Docker Hub, so we recommend regularly using the most recent version of our Docker image.

3.2.1 Building Docker Container

Run the following commands to build the docker container.

```
$ # Download Docker Image
$ docker pull mobilint/qbcompiler:v0.9.0.2
$ # mkdir {WORKING DIRECTORY} (if needed)
$ cd {WORKING DIRECTORY}
$ docker run -it --gpus all --ipc=host --name {YOUR_CONTAINER_NAME} -v ${pwd}:/workspace
  mobilint/qbcompiler:v0.9.0.2 /bin/bash
```

(Recommended) If the trained models and datasets are stored in different directories, you can mount them to the docker container as follows:

```
$ docker run -it --gpus all --ipc=host --name {YOUR_CONTAINER_NAME} -v ${pwd}:/workspace -v {PATH TO
  MODEL DIR}:/models -v {PATH TO DATASET DIR}:/datasets mobilint/qbcompiler:v0.9.0.2 /bin/bash
```

To run the NPU inside the Docker container, add --device /dev/aries0:/dev/aries0@ when creating the Docker container.

(Optional) Build the Docker image for the CPU-only version, allowing users to run the Mobilint Compiler on computers without a GPU.

```
$ # Download Docker Image
$ docker pull mobilint/qbcompiler:v0.9.0.2-cpu
$ cd {WORKING DIRECTORY}
```

3 Installation

```
$ docker run -it --ipc=host --name {YOUR_CONTAINER_NAME} -v $(pwd):/workspace  
mobilint/qbcompiler:v0.9.0.2-cpu /bin/bash
```

3.2.2 Installation of qubee

qubee compiler packages are available in [Mobilint® Software Development Kit \(SDK\)](#).

Run the following commands to install qubee on the docker container.

```
$ # Download qubee-0.9.0.2+aries-py3-none-any.whl file  
$ # Copy qubee whl file to Docker  
$ docker cp {Path to qubee-0.9.0.2+aries-py3-none-any.whl} {YOUR_CONTAINER_NAME}:/  
$ # Start Docker  
$ docker start {YOUR_CONTAINER_NAME}  
$ # Attach to Docker container  
$ docker exec -it {YOUR_CONTAINER_NAME} /bin/bash  
$ # Install qubee compiler  
$ cd /  
$ python -m pip install qubee-0.9.0.2+aries-py3-none-any.whl
```

The @.whl@ file includes the name of the required npu device, so make sure to check it before installation. Currently, we provide support for devices such as Aries, Aries2, and Regulus.

(Option, for WSL2) Run the following commands to install qubee on the docker container.

```
$ # Download qubee-0.9.0.2+aries-py3-none-any.whl file  
$ # Copy qubee whl file to Docker  
$ docker cp {Path to qubee-0.9.0.2+aries-py3-none-any.whl} {YOUR_CONTAINER_NAME}:/  
$ # Start Docker  
$ docker start {YOUR_CONTAINER_NAME}  
$ # Attach to Docker container  
$ docker exec -it {YOUR_CONTAINER_NAME} /bin/bash  
$ # Install qubee  
$ cd /  
$ python -m pip install qubee-0.9.0.2+aries-py3-none-any.whl
```

4. Tutorials

The tutorials below go through the steps for preparing the calibration dataset, model compiling, and inference.

4.1 Preparing Calibration Data

To compile the model, you should prepare the calibration dataset (the pre-processed inputs for the model) for quantization. There are three ways to make the calibration dataset as follows:

- (i) Pre-process the raw calibration dataset and save it as numpy tensors.
- (ii) Utilize a pre-processing configuration YAML file (only for images with **uniform format**).
- (iii) Use a manually defined pre-processing function (only for images with **uniform format**).

Important The process of making a calibration dataset may vary depending on whether you compile the model for CPU offloading or not. Currently, qubee compiles the model without CPU offloading by default. In this scenario, the pre-processed input shape should be in the format (H, W, C). On the other hand, when CPU offloading is employed, the pre-processed input shape should match the input shape that the original model takes.

4.1.1 Pre-process raw calibration dataset and save it as numpy tensors

You can save the pre-processed calibration dataset as numpy tensors with your custom pre-processing function and use them to compile the model.

An example code is shown below. The following code assumes that we have an image folder consisting of 1000 randomly selected JPEG image files from the [ImageNet](#) dataset for calibration prepared in directory /datasets/imagenet/cali_1000.

```
import os
import numpy as np
import cv2

def get_img_paths_from_dir(dir_path: str, img_ext = ["jpg", "jpeg", "png"]):
    assert os.path.exists(dir_path)
    candidates = os.listdir(dir_path)
    return [os.path.join(dir_path, y) for y in candidates if any([y.lower().endswith(e) for e in
img_ext])]

def pre_process(img_path: str, target_h: int, target_w: int):
    img = cv2.imread(img_path, cv2.IMREAD_COLOR)
    resized_img = cv2.resize(img, dsize=(target_w, target_h)).astype(np.float32)
    return resized_img

if __name__ == "__main__":
    img_dir = "/datasets/imagenet/cali_1000"
    save_dir = "/workspace/calibration/custom_single_input"
    target_h, target_w = 224, 224
    os.makedirs(save_dir, exist_ok=True)
    img_paths = get_img_paths_from_dir(img_dir)
    for i, img_path in enumerate(img_paths):
        fname = f"{i}.zfill(3) + ".numpy"
        fpath = os.path.join(save_dir, fname)
        x = pre_process(img_path, target_h, target_w)
        np.save(fpath, x)
```

The above results are in a directory containing the pre-processed calibration dataset (numpy tensors of shape (224,224, 3)), located at /workspace/calibration/custom_single_input.

4.1.2 Use a pre-processing configuration YAML file

Image pre-processing techniques such as resizing, cropping, and normalization are often applied in machine vision tasks. Users can construct a pre-processing configuration using a YAML file and prepare the calibration dataset via the API provided by qubee, `make_calib`. Please be aware that this method can only be employed when the raw data is an image. An example code is shown below. The following code assumes that images for calibration are prepared in the directory /workspace/cali_1000.

```
from qubee.calibration import make_calib
make_calib(
    args_pre="/workspace/mobilenet_v2.yaml", # path to pre-processing configuration yaml file
    data_dir="/datasets/imagenet/cali_1000", # path to folder of original calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed calibration data files
    save_name="mobilenet_v2", # tag for the generated calibration dataset
    max_size=50 # Maximum number of data to use for calibration
)
```

Yaml file example

```
# mobilenet_v2.yaml
Datatype: Image
GetImage:
  to_float32: false
  channel_order: RGB

Pre-Order: [ResizeTorch, CenterCrop, Normalize, SetOrder]
Pre-processing:
  ResizeTorch:
    size: [256, 256]
    interpolation: bilinear
  CenterCrop:
    size: [224, 224]
  Normalize:
    mean: [0.485, 0.456, 0.406]
    std: [0.229, 0.224, 0.225]
    to_float_div255: true
  SetOrder:
    shape: HWC
```

The above results are in a directory containing the pre-processed calibration dataset (numpy tensors), located at /workspace/calibration/mobilenet_v2. In addition, a calibration meta txt file containing the paths to the pre-processed numpy files is created, named /workspace/calibration/mobilenet_v2.txt.

Remark The sample dataset for calibration should be composed of images with the same format. If some are in color images and others are in grayscale images, the calibration dataset will not be created properly.

4.1.3 Use a manually defined pre-processing function

You can use your pre-processing function to make the calibration dataset via the API provided by qubee, `make_calib_man`. In this case, the pre-processing function should take the image path as input and return a numpy tensor. An example of the code is shown below. The following code assumes that images for calibration are prepared in the directory /datasets/imagenet/cali_1000.

```
import torch
```

```

import numpy as np
from PIL import Image
import torchvision.transforms.functional as F
from qubee.calibration import make_calib_man

def preprocess_resnet50(img_path: str):
    img = Image.open(img_path)
    resize_size=(232, 232)
    crop_size=(224, 224)
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    out = F.pil_to_tensor(img)
    out = F.resize(out, size=resize_size)
    out = F.center_crop(out, output_size=crop_size)
    out = out.to(torch.float, copy=False) / 255.
    out = F.normalize(out, mean, std)
    out = np.transpose(out.numpy(), axes=[1, 2, 0])
    return out

make_calib_man(
    pre_ftn=preprocess_resnet50, # callable function to pre-process the calibration data
    data_dir="/datasets/imagenet/cali_1000", # path to folder of original calibration data files such as images
    save_dir="/workspace/calibration/", # path to folder to save pre-processed calibration data files
    save_name="resnet50", # tag for the generated calibration dataset
    max_size=50 # Maximum number of data to use for calibration
)

```

The above results are in a directory containing the pre-processed calibration dataset (numpy tensors), located at /workspace/sample/calibration/resnet50. In addition, a calibration meta txt file containing the paths to the pre-processed numpy files is created, named /workspace/sample/calibration/resnet50.txt.

Remark Unless the custom pre-processing function contains proper exception handling, the sample dataset for calibration should be composed of images with the same format. Like the previous method, the calibration dataset will not be created properly if some are in color images and others are in grayscale images.

4.2 Compiling ONNX Models

ONNX is a recommended framework to be used for compiling the trained model. With simple code, the ONNX model can be directly parsed to obtain Mobilint IR. example code is shown below. The following code assumes that the calibration dataset and the model are prepared in the directory /workspace/calibration/resnet50 and /workspace/resnet50.onnx, respectively.

```

""" Compile ONNX model """
from qubee import mxq_compile
onnx_model_path = "/workspace/resnet50.onnx"
calib_data_path = "/workspace/calibration/resnet50"
# calib_data_path can be replaced with the path to the calibration meta file such as
# "/workspace/calibration/resnet50.txt"

mxq_compile(
    model=onnx_model_path,
    calib_data_path=calib_data_path,
    save_path="resnet50.mxq",
    backend="onnx"
)

```


4.3 Compiling PyTorch Models

PyTorch models can be compiled in two different ways. The first approach is converting the PyTorch model into the ONNX model with [torch.onnx](#) namespace, and compiling the converted model with the ONNX backend. The second approach is directly plugging the model into the Mobilint IR. Once the model is converted to Mobilint IR, then it is compiled into MXQ. The example code is shown below. The following codes assume that the calibration dataset is prepared in the directory `/workspace/calibration/resnet50`.

```

""" Compile PyTorch model """
from qubee import mxq_compile
### get resnet50 from torchvision
import torchvision
import torch
import numpy as np
calib_data_path = "/workspace/calibration/resnet50"
# A calibration meta file such as "/workspace/calibration/resnet50.txt" can be used instead.

### get resnet50 from torchvision and convert it to torchscript
torch_model = torchvision.models.resnet50(pretrained=True)
torchscript_model_path = "/workspace/resnet50.pt"

dummy_input = np.random.randn(1,3,224,224).astype(np.float32)
feed_dict = {"input": dummy_input}
#When compiling with TorchScript, feed_dict information is required, which contains the temporary input name and a sample input.
scripted_model = torch.jit.script(torch_model, torch.tensor(dummy_input))
torch.jit.save(scripted_model, torchscript_model_path)

mxq_compile(
    model=torchscript_model_path,
    calib_data_path=calib_data_path,
    backend="torchscript",
    save_path="resnet50.mxq",
    feed_dict=feed_dict,
)

```

4.4 Compiling TensorFlow/Keras Models

Since Keras works as an interface for TensorFlow, models on the Keras framework can be converted to Mobilint IR via TensorFlow. Currently, the qubee compiler supports TensorFlow models saved in the format of the SavedModel or frozen graph. For the SavedModel format, which includes the serialized model ending with `.pb`, the model can be directly compiled. For the frozen graph, the compiler requires the input node name and the output node name, which can be viewed by [Netron](#). The following codes assume the calibration dataset is prepared in the directory `/workspace/calibration/resnet50`.

```

""" Compile Keras/TensorFlow model in SavedModel format """
from qubee import mxq_compile
import tensorflow as tf

keras_model = tf.keras.applications.resnet50.ResNet50() # Load a Keras model
calib_data_path = "/workspace/calibration/resnet50"
# A calibration metadata file such as "/workspace/calibration/resnet50.txt" can be used instead.

keras_model_save_path = "/workspace/tf_models/resnet50.h5" # path to save .h5 keras model
keras_model.save(keras_model_save_path)

mxq_compile(
    model=keras_model_save_path,

```

```
calib_data_path=calib_data_path,  
backend="tf",  
save_path="resnet50.mxq",  
)
```

4.5 Compiling TensorFlow Lite Models

The qubee compiler supports TensorFlow Lite models. With the given TensorFlow Lite model, the calibration dataset, and the backend, the model can be compiled into Mobilint IR. The following code assumes the calibration dataset is prepared in the directory `/workspace/calibration/resnet50`.

```
""" Compile Tensorflow Lite model """  
from qubee import mxq_compile  
import tensorflow as tf  
  
keras_model = tf.keras.applications.resnet50.ResNet50() # Load a pre-trained Keras model  
calib_data_path = "/workspace/calibration/resnet50"  
# A calibration metadata file such as "/workspace/calibration/resnet50.txt" can be used instead.  
  
keras_model_save_path = "/workspace/tf_models/resnet50.h5" # path to save .h5 keras model  
tflite_model_path = "/workspace/tflite_models/resnet50.tflite"  
keras_model.save(keras_model_save_path)  
  
loaded_keras_model = tf.keras.models.load_model(keras_model_save_path)  
tflite_model = tf.lite.TFLiteConverter.from_keras_model(loaded_keras_model).convert() # Convert the model to  
TFLite format  
with open(tflite_model_path, 'wb') as f:  
    f.write(tflite_model)  
  
mxq_compile(  
    model=tflite_model_path,  
    calib_data_path=calib_data_path,  
    backend="tflite",  
    save_path="resnet50.mxq",  
)
```

5. CPU Offloading (Beta Version)

Remark To proceed inference with CPU offloading, it requires a runtime library that supports the MXQ file that is compiled for CPU offloading.

From qubee v0.7, we provide a Beta version of CPU offloading for MXQ compilation. CPU offloading makes it easier for users to compile their models by automatically offloading the computation that Mobilint NPU does not support to the CPU. For example, if a pre-processing or post-processing included in the model involves operations that the NPU does not support, the user would have to implement them manually after compiling, but CPU offloading covers most of these operations and eliminates the need for additional work.

When CPU offloading is employed, the procedures for preparing the calibration dataset and compiling the model vary slightly as follows:

(i) The pre-processed input shape should match the original model's input shape, whereas the pre-processed input shape should be in the format (H, W, C) to compile the model without CPU offloading.

(ii) Set the argument `cpu_offload` of function `mxq_compile` True to enable CPU offloading.

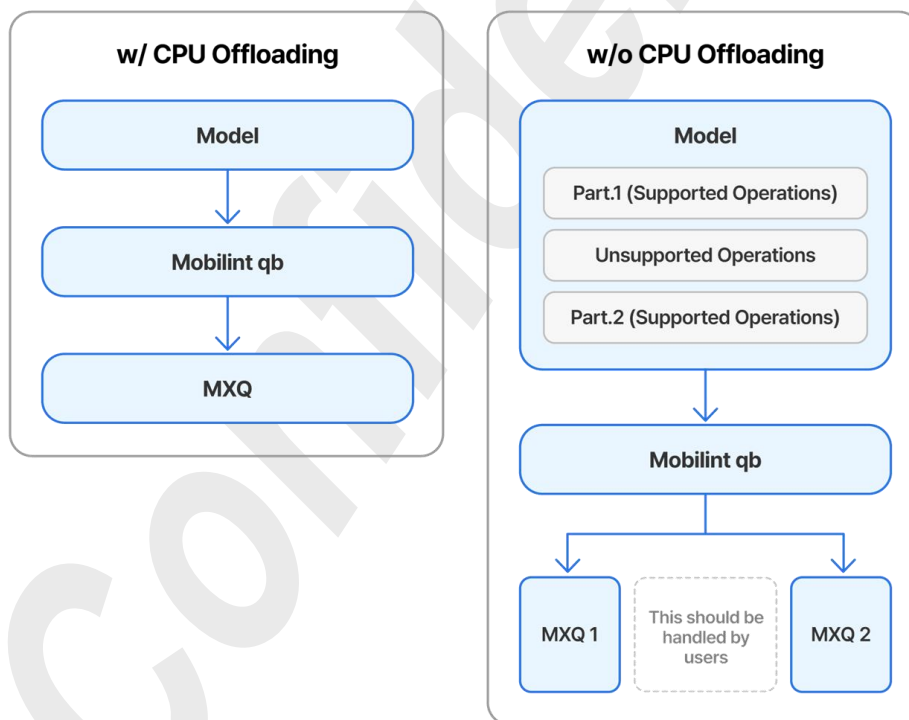


Figure 5-1. SDK CPU Offloading

6. Supported Frameworks

We support almost all the commonly used Machine Learning frameworks & libraries such as ONNX, PyTorch, Keras, TensorFlow, and TensorFlow Lite.

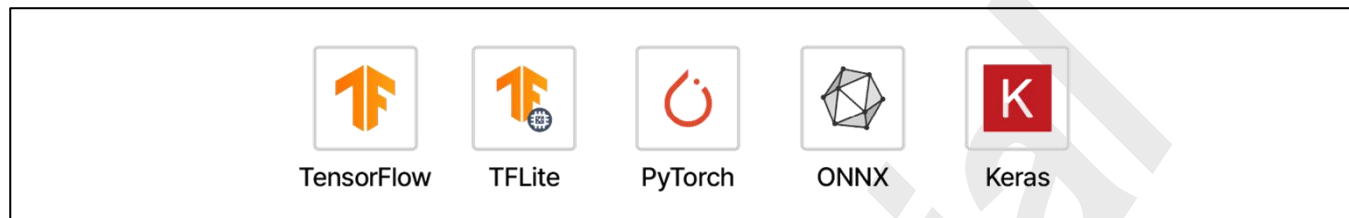


Figure 6-1. Supported deep-learning frameworks

6.1 Supported Operations (ONNX)

Confidential

Table 6-1. ONNX Supported Operations

API Name	Comments
Add	Broadcast only for specific cases of constant addition: Adding scalar, Adding channel-size vector.
And	
ArgMax	
AveragePool	Only dilation=1, count_include_pad=1.
BatchNormalization	Only training_mode=0.
Cast	
Ceil	
Clip	
Concat	Only along channel axis.
Constant	
ConstantOfShape	
Conv	
ConvTranspose	
DepthToSpace	
Div	Only constant division. Support broadcast same as Add.
Elu	
Equal	
Erf	
Exp	
Expand	
Flatten	Only axis=1 and before fully connected layer or Conv w/ 1x1 kernel.
Floor	
Gather	
GatherND	
Gemm	Only transA=0. Only for the following specific case: Input A is a flatten activation and input B is 2D tensor.
GlobalAveragePool	
Greater	
HardSigmoid	
HardSwish	
Identity	
InstanceNormalization	
LayerNormalization	
LeakyRelu	
Less	
Loop	

MatMul	Only for the following specific case: Input A is a flatten activation and input B is 2D tensor or vice-versa.
Max	
MaxPool	Only dilation=1.
Min	
Mod	
Mul	Only constant multiplication. Support broadcast same as Add.
NonMaxSuppression	
NonZero	
Not	
Or	
Pad	Constant, reflect, edge modes are supported
Pow	
PRelu	
Range	
Reciprocal	
ReduceMax	Only along height and width.
ReduceMean	Only along height and width.
ReduceMin	Only along height and width.
ReduceProd	Only along height and width.
ReduceSum	Only along height and width.
Relu	Only scalar slope.
Reshape	Only channel-wise flatten and before fully connected layer or Conv w/ 1x1 kernel. Only allowzero=0.
Resize	Only for the following specific case: Only mode = "nearest" and coordinate_transformation_mode = "half_pixel" or "pytorch_half_pixel", Only mode = "linear" and coordinate_transformation_mode = "half_pixel" or "pytorch_half_pixel", Attributes axes, antialias, keep_aspect_ratio_policy nearest_mode are not supported.
ScatterND	
Shape	
Sigmoid	
Slice	Only channel-wise slice.
Softmax	
Softplus	
Split	
Sqrt	
Squeeze	Only when resulting tensor has 2D shape. Squeeze along batch axis is unsupported.

Sub	Support broadcast same as Add.
Tanh	
Tile	Batch-wise tile is unsupported.
TopK	
Transpose	Only for the following specific case: Transpose-Flatten-Linear.
Unsqueeze	
Upsample	Only mode "nearest" and "linear".
Xor	

6.2 Supported operations (PyTorch)

Remark Since the Torchscript backend framework is based on [Torchscript-Based-ONNX-Exporter](#), even if the operation is not listed below, it may be supported if it has corresponding ONNX operation, which is supported by qubee.

Table 6-2. PyTorch Supported Operations

API Name	Comments
ADD	Only alpha=1. See supported operations (ONNX): Add
AMAX	See supported operations (ONNX): ReduceMax
AMIN	See supported operations (ONNX): ReduceMin
ARGMAX	See supported operations (ONNX): ArgMax
CAT	Only along channel axis. See supported operations (ONNX): Concat
CEIL	See supported operations (ONNX): Ceil
CLAMP	See supported operations (ONNX): Clip
DIV	Only constant division. See supported operations (ONNX): Div
EQ	See supported operations (ONNX): Equal
ERF	See supported operations (ONNX): Erf
EXP	See supported operations (ONNX): Exp
FLOOR	See supported operations (ONNX): Floor
FMOD	See supported operations (ONNX): Mod
GATHER	See supported operations (ONNX): GatherND
GT	See supported operations (ONNX): Greater
LOGICAL_AND	See supported operations (ONNX): And
LOGICAL_NOT	See supported operations (ONNX): Not
LOGICAL_OR	See supported operations (ONNX): Or
LOGICAL_XOR	See supported operations (ONNX): Xor
LT	See supported operations (ONNX): Less
MATMUL	See supported operations (ONNX): MatMul
MAX	See supported operations (ONNX): Max
MEAN	See supported operations (ONNX): ReduceMean
MIN	See supported operations (ONNX): Min
MUL	Only constant multiplication. See supported operations (ONNX): Mul
ADAPTIVEAVGPOOL2D	See supported operations (ONNX): AveragePool
ADAPTIVEMAXPOOL2D	See supported operations (ONNX): MaxPool
AVGPOOL2D	Only dilation=1, count_include_pad=1. See supported operations (ONNX): AveragePool
BATCHNORM2D	See supported operations (ONNX): BatchNormalization
CONV2D	See supported operations (ONNX): Conv
CONVTRANSPOSE2D	See supported operations (ONNX): ConvTranspose
ELU	See supported operations (ONNX): Elu
FLATTEN	Only channel-wise flatten and before fully connected layer or Conv w/ 1x1 kernel. See supported operations (ONNX): Flatten

INTERPOLATE	See supported operations (ONNX): Resize.
PAD	See supported operations (ONNX): Pad
HARDSIGMOID	See supported operations (ONNX): HardSigmoid
HARDSWISH	See supported operations (ONNX): HardSwish
IDENTITY	See supported operations (ONNX): Identity
INSTANCENORM2D	See supported operations (ONNX): InstanceNormalization
LEAKYRELU	See supported operations (ONNX): LeakyRelu
LINEAR	See supported operations (ONNX): Gemm
MAXPOOL2D	Only dilation=1. See supported operations (ONNX): MaxPool
PRELU	See supported operations (ONNX): PRelu
RELU	See supported operations (ONNX): Relu
SIGMOID	See supported operations (ONNX): Sigmoid
SOFTMAX	See supported operations (ONNX): Softmax
SOFTPLUS	Only beta=1. See supported operations (ONNX): Softplus
TANH	See supported operations (ONNX): Tanh
UPSAMPLE	Only mode "nearest" and "linear". Only scales=[2,2]. See supported operations (ONNX): Upsample
PERMUTE	See supported operations (ONNX): Transpose
POW	See supported operations (ONNX): Pow
PROD	See supported operations (ONNX): ReduceProd
RECIPROCAL	See supported operations (ONNX): Reciprocal
RESHAPE	Only channel-wise flatten and before fully connected layer or Conv w/ 1x1 kernel. See supported operations (ONNX): Reshape
SCATTER	See supported operations (ONNX): ScatterND
SPLIT	See supported operations (ONNX): Split
SQRT	See supported operations (ONNX): Sqrt
SQUEEZE	Only when resulting tensor has 2D shape. Squeeze along batch axis is unsupported. See supported operations (ONNX): Squeeze
SUB	Only alpha=1. See supported operations (ONNX): Sub
TENSOR	See supported operations (ONNX): Constant
TILE	Batch-wise tile is unsupported. See supported operations (ONNX): Tile
TOPK	See supported operations (ONNX): TopK
TRANSPOSE	Only before fully connected layer. See supported operations (ONNX): Transpose
UNSQUEEZE	See supported operations (ONNX): Unsqueeze

6.3 Supported operations (TensorFlow/Keras/TensorFlow Lite)

As mentioned in the previous section, Keras works as an interface for TensorFlow 2, and they save the model in the same format as the frozen graph, which ends with .pb. Therefore, the TensorFlow/Keras/TensorflowLite operation is supported if it can be described by the TensorFlow raw operations listed below when the model is saved in the format of the frozen graph.

Confidential

Table 6-3. TensorFlow Supported Operations

API Name	Comments
Placeholder	According to the official document, this operation will fail with an error if it is executed.
PlaceholderWithDefault	
Floor	
Identity	
Const	
IdentityN	
Pad	
PadV2	
Conv2D	
DepthwiseConv2dNative	
Conv2DBackpropInput	
MatMul	
FusedBatchNormV3	
FusedBatchNorm	
MaxPool	
AvgPool	
Mean	Only along height, width, and channel.
ResizeNearestNeighbor	
ResizeBilinear	
ConcatV2	Only along channel axis.
Add	
AddV2	
Mul	Only constant multiplication.
Sub	
RealDiv	Only constant division.
AddN	
BiasAdd	
Relu	
Sigmoid	
Softplus	
Exp	
Tanh	
Neg	
LeakyRelu	
Relu6	
Softmax	
ArgMax	
Switch	

6 Supported Frameworks

Merge	
Shape	
Reshape	Only channel-wise flatten and before fully connected layer or Conv w/ 1x1 kernel.
Transpose	Only before fully connected layer.
ExpandDims	
Squeeze	Only when resulting tensor has 2D shape. Squeeze along batch axis is unsupported.
StridedSlice	ellipsis_mask, new_axis_mask, shrink_axis_mask are unsupported.
Slice	
Pack	
Split	
SplitV	
Range	
Fill	
Tile	Batch-wise tile is unsupported.
Cast	
TensorArrayV3	
Maximum	
Sqrt	
Rsqrt	
Rint	
Greater	
LogicalAnd	
Equal	
GreaterEqual	
RandomUniform	
NoOp	
Assert	
ReadFile	
DecodeJpeg	

7. API Reference

7.1 Function: mxq_compile

Compile a given model directly without creating an instance of "Model_Dict".

Confidential

Table 7-1. mxq_compile

Parameter	Type	Description
model	string or model instance	Model path or model instance. Model should be instance for the following cases: When using backend="onnx", it should be the path to ONNX model file When using backend="torchscript", it should be the path to PyTorch model file When using backend="tf", it should be the path to the folder saving TensorFlow PB graph and assets. When using backend="tflite", it should be the path to TF Lite model file.
calib_data_path	string	A path to the calibration dataset. It can be either of a path to the text (or json) file containing the paths to the pre-processed numpy files or a directory containing the pre-processed numpy files.
save_subgraph_type	int (optional)	0: No subgraph is saved, and the .mblt file is not generated. 1: Only the graph structure is saved. 2: Both the graph structure and weights are saved.
output_subgraph_path	string (optional)	Specifies the file path where the .mblt file will be saved. This file is only generated if save_subgraph_type is set to either 1 or 2. The saved file can be used for visualization purposes.
model_nickname	string (optional)	Model nickname used in qubee. It is used in qubee to facilitate quicker recompilation of the same models. Qubee stores prior optimization information under this nickname, enabling it to locate and utilize the previously compiled results for faster processing. It is auto-generated from the model's base name, if not provided. For instance, a model "/workspace/onnx/resnet50.onnx" results in "resnet50". If not derivable, "temporary" is the default nickname.
save_path	string (optional)	Filename of the resulting .mxq. If it is None, then it is set to "model_nickname".mxq Defaults to None.
feed_dict	dict (optional)	Example input for input shape and inference value. Moreover, It can be used for torchscript compilation.
in_dformats	dict (optional)	Example dataformat of the model for additional inputs information.
backend	string (optional)	Which framework to use to get the Mobilint IR. It must be one of "onnx", "tf", "tflite", and "torchscript". They correspond to deep learning frameworks as follows: "onnx": ONNX, "tf": TensorFlow, "tflite": TensorFlow Lite, "torchscript": PyTorch

		Defaults to "onnx".
device	string (optional)	Device to be used for compile and inference. Either "cpu" or "gpu". Defaults to "cpu".
save_sample	bool (optional)	Create the 'sampleInOut' folder in the current directory and store the input and output binary files in it if True. This is for debugging. Defaults to False.
yolo_decode_include (beta)	bool (optional)	An option to configure whether the YOLO decode process is executed on the NPU or not.
inference_scheme	string (optional)	The type of inference used for running on an NPU. This option must be one of the following: single, multi, or global.
quantize_method	string (optional)	Quantization method to determine the scale parameter in the quantization. Currently, "Max", "Percentile", "MaxPercentile", "FastPercentile", and "KL" are supported. Defaults to "Percentile".
quantize_percentile	float (optional)	Percentile used for the quantization method "Percentile", "MaxPercentile", and "FastPercentile". This should be between 0 and 1. (Ex. 0.999, 0.9999) Defaults to 0.99995.
topk_ratio	float (optional)	It is used for quantization method "maxpercentile". Defaults to 0. The larger this value is, the more data is used for calibration. This should be between 0 and 1, but using a value of 0.01 or less is recommended.
smooth_factor	float (optional)	Smooth factor for Gaussian kernel construction, which is required on KL divergence estimation. Defaults to 1.6.
is_quant_ch	bool (optional)	Use multi-channel quantization if True. Defaults to False.
is_asym_quant	bool (optional)	Use asymmetric quantization if True. Default to False
save_sample	bool (optional)	If True, create the "sampleInOut" folder in the current directory and store the input and output binary files in it. Defaults to False.
singlecore_compile	bool (optional)	Use singlecore to use large dram memory
use_random_calib	bool (optional)	If True, it compiles the given model with random calibration data. This is just used to check if the model is compilable without making a calibration data. Defaults to False.
cpu_offload	bool (optional)	Use CPU offloading for NPU inference if True. Defaults to False.
quant_output	string (optional)	Quantization method that applied to the output layer. "layer", "ch" and "sigmoid" options are available. If "layer", per-layer quantization is applied to the output layer. If is_quant_ch is true, then the computed

		quantization scale for each channel of the output layer will be merged into single value. If "ch", per-channel quantization is applied to the output layer. This option is valid only when is_quant_ch is true. If "sigmoid", assign quantization scale that computed with sigmoid function. Defaults to "layer".
sample_dtype	string (optional)	Data type for saving the sample inference result. It should be either "float" or "int8".
example_input	dict (optional)	Dictionary to provide example input to nodes which have more than one unknown dimensions. (Only for ONNX/PyTorch/TensorFlow Frameworks)
feed_dict	dict (optional)	Dictionary to provide input and output node information when loading the model that saved in format of the frozen graph. (For TensorFlow) Dictionary to provide shape (and value) information of the input and output to validate proper compilation. (For Tensorflow Lite)
adaq_useadaquant	bool (optional)	If True, enable the finetuning with AdaQuant after quantization. Defaults to False.
adaq_weightDeltaLR	float (optional)	Learning rate for finetuning weight delta(weight update) of AdaQuant. (Recommend: 1e-6 ~ 5e-5) Defaults to 0.
adaq_biasDeltaLR	float (optional)	Learning rate for finetuning bias delta(bias update) of AdaQuant. (Recommend: weightDeltaLR/10 ~ weightDeltaLR/2) Defaults to 0.
adaq_weightScaleLR	float (optional)	Learning rate for finetuning weight quantization scale of AdaQuant. Defaults to 0.
adaq_biasScaleLR	float (optional)	Learning rate for finetuning bias quantization scale of AdaQuant. Defaults to 0.
adaq_actScaleLR	float (optional)	Learning rate for finetuning activation quantization scale of AdaQuant. Defaults to 0.
adaq_batchSize	int (optional)	Batch size for running AdaQuant. Defaults to 16.
adaq_epoch	int (optional)	Epochs for repeating AdaQuant update. Defaults to 10.
act_scale_min	float (optional)	Minimum value for activation quantization scale. Defaults to 0.0078125.
weight_scale_min	float (optional)	Minimum value for weight quantization scale. Defaults to 1e-6.
hist_numCluster	int (optional)	Number of clusters for histogramobserver. Defaults to 4.
hist_searchPercentileMin	float (optional)	Minimum searching percentile for histogramobserver.

		Defaults to 0.9999.
hist_epsilon	float (optional)	Minimum bin size for histogramobserver. Prevents GPU explosion. Defaults to 0.01.
hist_adjustCoeff	float (optional)	Adjust coefficient for histogramobserver. Defaults to 0.5.
preprocess_dict	int (optional)	Preprocess information
MOD_apply	bool (optional)	Run Min output difference (MOD) QAT if True
MOD_epochs	int (optional)	Epochs for MOD QAT
MOD_actScaleLR	float (optional)	LR of activation scale for MOD QAT
MOD_zeropointLR	float (optional)	LR of zeropoint for MOD QAT
MOD_weightScaleLR	float (optional)	LR of weight scale for MOD QAT
MOD_weightLR	float (optional)	LR of weight for MOD QAT
MOD_biasLR	float (optional)	LR of bias for MOD QAT
MOD_batchSize	int (optional)	Batch size for MOD QAT
MOD_qDrop	float (optional)	QDrop ratio for MOD QAT. range=[0, 1]
MOD_quantizeWeight	bool (optional)	Use quantized weight for MOD QAT if True
MOD_weightScaleInit	string (optional)	Weight quantization init method for for MOD QAT. Options=('MinMax')
MOD_weightScaleInit	string (optional)	Weight quantization init method for for MOD QAT. Options=('MinMax')
MOD_downresolMode	string (optional)	Down-resolution of fake-quantization, Options=('STE', 'NIPQ')
MOD_post	string (optional)	Post-processing function to make the output values, Options=(AnchorlessDetection', 'AnchorlessDetectionNMS', 'AnchorlessDetectionNMSCIoU')
MOD_boxConfThres	float (optional)	Confidence threshold of NMS if NMS is included in the post-processing.
MOD_boxIoUThres	float (optional)	IoU threshold of NMS if NMS is included in the post-processing.
MOD_useOutputs	bool (optional)	Compare only intermediate layers if True.
MOD_KLTemperature	float (optional)	Temperature of KL divergence loss if it is used.
MOD_reconProb	float (optional)	Sampling probability of intermediate layers. range=[0, 1]
MOD_reconCoeff	float (optional)	Coefficient to the loss of intermediate layers.
MOD_lambda_0	float (optional)	1st coeff of AnchorlessDetection loss
MOD_lambda_1	float (optional)	2nd coeff of AnchorlessDetection loss
MOD_lambda_2	float (optional)	3rd coeff of AnchorlessDetection loss
MOD_seed	int (optional)	Random seed for MOD QAT

7.1.1 Tips for choosing quantization methods and selecting arguments

"Percentile", "MaxPercentile", and "FastPercentile" quantization methods take a hyperparameter called *percentile*. An increase in this value corresponds to a broader quantization interval. To elaborate further, a higher *percentile* results in reduced overflow, albeit at the expense of accuracy.

The "Percentile" method computes the quantization interval by taking samples of certain percentiles from the histogram of the calibration data stack. The "MaxPercentile" method determines the quantization interval using the data stack that consists of the min/max values of the calibration data, and the "FastPercentile" method determines the quantization interval using the data stack that consists of the reduced and smoothed samples of the calibration data. As a result, the "Percentile" method, which is the slowest, requires the highest *percentile* value, while the "FastPercentile" method, which is faster, requires the lower *percentile* value. The "MaxPercentile" method, which is the fastest, requires the lowest *percentile* value in a general sense.

On the hyperparameter tuning step, we recommend choosing an initial value of 0.9999 to 0.999999 for the "Percentile" method, 0.99 to 0.99999 for the "FastPercentile" method, and 0.9 to 0.9999 for the "MaxPercentile" method.

The "is_quant_ch" argument enables channel-wise quantization. When set to True, the quantization is performed on a per-channel basis. This method is particularly useful for models, in which activations vary significantly across channels. However, it may take a longer time to compile the model.

The "quant_output" argument is used to determine the quantization method for the output layer. When the original model's output is various across the channels, it is recommended to set "ch" to keep channel-wise quantization. Otherwise, set "layer" to quantize the output layer as a whole.

7.1.2 Compiling Models with Custom Input (ONNX/Torch/TensorFlow)

When the model lacks input shape information in more than one dimension, qubee may generate the error. Therefore, you should provide a numpy input array along with the model during compilation to infer the unknown shape and input dataformats. For example, if your model has three inputs named <input1, input2, input3> and the shape of input1 is unknown, then you need to prepare the numpy array for input1 with the custom shape user provided. For input2 and input3, use the original source shape already provided.

```
from qubee import mxq_compile
import numpy as np

example_input = {"input_node_name_1": np.random.randn(1, 3, 224, 224).astype(np.float32), # Custom shape
                 "input_node_name_2": np.random.randn(1, 8, 224, 224).astype(np.float32), # Original source
                 "input_node_name_3": np.random.randn(1, 4, 56, 56).astype(np.float32)} # Original source shape
                 specified by user
                 shape already given
                 already given

in_dformats = {"input_node_name_1": "NCHW",
               "input_node_name_2": "NCHW",
               "input_node_name_3": "NCHW",}

onnx_model_path = "path/to/your/model.onnx"
mxq_compile(
    model=onnx_model_path,
    feed_dict=feed_dict,
    in_dformats=in_dformats,
    backend="onnx"
)
```

In this example, feed_dict is a dictionary where each key corresponds to an input node name of the model, and each value is a numpy array with the appropriate shape. in_dformats contains dataformat information to support parsing process. The input for input_node_name_1 has a custom shape, while the inputs for input_node_name_2 and input_node_name_3 use their original source shapes.

7.2 Class: Model_Dict

This class supports parsing and compiling deep learning frameworks into .mxq files. The executed .mxq file can be used with the Maccel runtime library.

Table 7-2. Model_Dict Class

Parameter	Type	Description
model	string or model class of the corresponding framework	Model path or model instance. The following cases are supported: When using backend="onnx", it should be the path to ONNX model file When using backend="torchscript", it should be the path to PyTorch model file When using backend="tf", it should be the path to the folder saving TensorFlow PB graph and assets, or the frozen graph. When using backend="tflite", it should be the path to TF Lite model file.
backend	string (optional)	Which framework to use to get the Mobilint IR. It must be one of "onnx", "tf", "tflite", or "torchscript". They correspond to deep learning frameworks as follows: "onnx": ONNX, "tf": TensorFlow, "tflite": TensorFlow Lite, "torchscript": PyTorch Defaults to "onnx".
device	string (optional)	Device to be used for compile and inference. Either "cpu" or "gpu". Defaults to "cpu".
target_device	string (optional)	A target device name, such as "aries" or "regulus," is automatically determined by a library file for high-level parsing.
feed_dict	dict (optional)	Dictionary to provide example input to nodes which have more than one unknown dimensions. It is used to support parsing processes.
in_dformats	dict (optional)	Data format information in the form of {input name: "NCHW"} is used to additionally support the parsing process.
parser_config	ParserConfig	A configuration for adjusting options in the high-level parsing process is provided, though users are advised to use the default settings.

7.2.1 Methods

Table 7-3. Model_Dict Methods

Methods	Description
<code>__init__</code>	Constructor of Mobilint IR model.
<code>compile</code>	Compile the given model into MXQ format.
<code>parse</code>	The parsing process corresponding to the model is carried out in Python. It is automatically executed when the compile function is run.
<code>parser_inferences</code>	Floating inference with the Mobilint IR. This can be used to check the built IR returns the same output as the model.
<code>clear_temp_dir</code>	Clear temporary directory.
<code>save_mblt</code>	A serialized file containing the graph structure and weight information to be passed to the backend compiler is saved.
<code>get_btype_data</code>	Serializes the graph structure and weight information to produce byte data.
<code>get_body_md_wd</code>	A function that extracts information from the largest supported graph among the parsed graphs. It is automatically used during the parsing process when the compile function is executed with <code>cpu-offload</code> set to <code>off</code> .
<code>get_target_device_name</code>	Determines the target device, such as "aries" or "regulus." The selection is automatically handled through the library file.

7.2.2 Method Details

Confidential

Table 7-4. Model_Dict.compile

Parameter	Type	Description
calib_data_path	string	A path to the calibration dataset. It can be either of a path to the text (or json) file containing the paths to the pre-processed numpy files or a directory containing the pre-processed numpy files.
save_path	string	Filename of the resulting .mxq.
save_subgraph_type	int (optional)	0: No subgraph is saved, and the .mblt file is not generated. 1: Only the graph structure is saved. 2: Both the graph structure and weights are saved.
output_subgraph_path	string (optional)	Specifies the file path where the .mblt file will be saved. This file is only generated if save_subgraph_type is set to either 1 or 2. The saved file can be used for visualization purposes.
model_nickname	string (optional)	Model nickname used in qubee. It is used in qubee to facilitate quicker recompilation of the same models. Qubee stores prior optimization information under this nickname, enabling it to locate and utilize the previously compiled results for faster processing. It is auto-generated from the model's base name, if not provided. For instance, a model "/workspace/onnx/resnet50.onnx" results in "resnet50". If not derivable, "temporary" is the default nickname.
save_msgpack_name	string (optional)	The name used when utilizing the compile_ndc function.
compile_ndc	bool (optional)	It compiles up to the NDC stage only.
yolo_decode_include (beta)	bool (optional)	An option to configure whether the YOLO decode process is executed on the NPU or not.
inference_scheme	string (optional)	The type of inference used for running on an NPU. This option must be one of the following: single, multi, or global.
quantize_method	string (optional)	Quantization method to determine the scale parameter in the quantization. Currently, "Max", "Percentile", "MaxPercentile", "FastPercentile", and "KL" are supported. Defaults to "Percentile".
quantize_percentile	float (optional)	Percentile used for the quantization method "Percentile", "MaxPercentile", and "FastPercentile". This should be between 0 and 1. (Ex. 0.999, 0.9999). Defaults to 0.9999.
topk_ratio	float (optional)	It is used for quantization method

		"maxpercentile". Defaults to 0. The larger this value is, the more data is used for calibration. This should be between 0 and 1, but using a value of 0.01 or less is recommended.
smooth_factor	float (optional)	Smoothing factor that is required for Gaussian kernel construction on KL divergence estimation. Defaults to 1.6.
is_quant_ch	bool (optional)	Use multi-channel quantization if True. Defaults to False.
save_sample	bool (optional)	If True, create the "sampleInOut" folder in the current directory and store the input and output binary files in it. Defaults to False.
use_random_calib	bool (optional)	If True, it compiles the given model with random calibration data. This is just used to check if the model is compilable without making a calibration data. Defaults to False.
cpu_offload	bool (optional)	Use CPU offloading for NPU inference if True. Defaults to False.
quant_output	string (optional)	Quantization method that applied to the output layer. "layer", "ch" and "sigmoid" options are available. If "layer", per-layer quantization is applied to the output layer. If is_quant_ch is true, then the computed quantization scale for each channel of the output layer will be merged into single value. If "ch", per-channel quantization is applied to the output layer. This option is valid only when is_quant_ch is true. If "sigmoid", assign quantization scale that computed with sigmoid function. Defaults to "layer".
sample_dtype	string (optional)	Data type for saving the sample inference result. It should be either "float" or "int8".
adaq_useadaquant	bool (optional)	If True, enable the finetuning with AdaQuant after quantization. Defaults to False.
adaq_weightDeltaLR	float (optional)	Learning rate for finetuning weight delta(weight update) of AdaQuant. (Recommend: 1e-6 ~ 5e-5) Defaults to 0.
adaq_biasDeltaLR	float (optional)	Learning rate for finetuning bias delta(bias update) of AdaQuant. (Recommend: weightDeltaLR/10 ~ weightDeltaLR/2) Defaults to 0.
adaq_weightScaleLR	float (optional)	Learning rate for finetuning weight quantization scale of AdaQuant. Defaults to 0.
adaq_biasScaleLR	float (optional)	Learning rate for finetuning bias quantization

		scale of AdaQuant. Defaults to 0.
adaq_actScaleLR	float (optional)	Learning rate for finetuning activation quantization scale of AdaQuant. Defaults to 0.
adaq_batchSize	int (optional)	Batch size for running AdaQuant. Defaults to 16.
adaq_epoch	int (optional)	Epochs for repeating AdaQuant update. Defaults to 10.
act_scale_min	float (optional)	Minimum value for activation quantization scale. Defaults to 0.0078125.
weight_scale_min	float (optional)	Minimum value for weight quantization scale. Defaults to 1e-6.
hist_numCluster	int (optional)	Number of clusters for histogramobserver. Defaults to 4.
hist_searchPercentileMin	float (optional)	Minimum searching percentile for histogramobserver. Defaults to 0.9999.
hist_epsilon	float (optional)	Minimum bin size for histogramobserver. Prevents GPU explosion. Defaults to 0.01.
hist_adjustCoeff	float (optional)	Adjust coefficient for histogramobserver. Defaults to 0.5.
preprocess_dict	int (optional)	Preprocess information
MOD_apply	bool (optional)	Run Min output difference (MOD) QAT if True
MOD_epochs	int (optional)	Epochs for MOD QAT
MOD_actScaleLR	float (optional)	LR of activation scale for MOD QAT
MOD_zeropointLR	float (optional)	LR of zeropoint for MOD QAT
MOD_weightScaleLR	float (optional)	LR of weight scale for MOD QAT
MOD_weightLR	float (optional)	LR of weight for MOD QAT
MOD_biasLR	float (optional)	LR of bias for MOD QAT
MOD_batchSize	int (optional)	Batch size for MOD QAT
MOD_qDrop	float (optional)	QDrop ratio for MOD QAT. range=[0, 1]
MOD_quantizeWeight	bool (optional)	Use quantized weight for MOD QAT if True
MOD_weightScaleInit	string (optional)	Weight quantization init method for for MOD QAT. Options=('MinMax')
MOD_weightScaleInit	string (optional)	Weight quantization init method for for MOD QAT. Options=('MinMax')
MOD_downresolMode	string (optional)	Down-resolution of fake-quantization, Options=('STE', 'NIPQ')
MOD_post	string (optional)	Post-processing function to make the output values, Options=('AnchorlessDetection', 'AnchorlessDetectionNMS', 'AnchorlessDetectionNMSCIoU')
MOD_boxConfThres	float (optional)	Confidence threshold of NMS if NMS is included in the post-processing.

MOD_boxIoUThres	float (optional)	IoU treshold of NMS if NMS is included in the post-processing.
MOD_useOutputs	bool (optional)	Compare only intermediate layers if True.
MOD_KLTemperature	float (optional)	Temperature of KL divergence loss if it is used.
MOD_reconProb	float (optional)	Sampling probability of intermediate layers. range=[0, 1]
MOD_reconCoeff	float (optional)	Coefficient to the loss of intermediate layers.
MOD_lambda_0	float (optional)	1st coeff of AnchorlessDetection loss
MOD_lambda_1	float (optional)	2nd coeff of AnchorlessDetection loss
MOD_lambda_2	float (optional)	3rd coeff of AnchorlessDetection loss
MOD_seed	int (optional)	Random seed for MOD QAT

7.3 Function: make_calib

From the given images and preprocessing configuration, create the preprocessed numpy files and a txt file containing their paths.

Table 7-5. make_calib

Parameter	Type	Description
args_pre	string or Dict	Path to a Yaml file or dictionary containing preprocessing configuration information. Refer to 7.4. for details.
data_dir	string	Directory of data to be used for calibration.
save_dir	string	Directory to save the pre-processed numpy files and txt file which contains their paths.
save_name	string (optional)	Name for resulting files. Numpy files will be saved under {save_dir}/{save_name}_npz directory. Text file will be saved in {save_dir}/{save_name}.txt. If it is not provided, it is set to the basename of data_dir.
anno_json	string (optional)	Path to an annotation json file for COCO format. When provided, make_calib function randomly selects samples considering class balance. Defaults to None.
file_format	string (optional)	Filename format using image_idx. Defaults to '%012d.jpg'.
max_size	int (optional)	Maximum size of the resulting calibration data. Defaults to -1, which means no limit on the number of the calibration data.
remove_npy	bool (optional)	If True, remove pre-existing numpy files. Defaults to False.
seed	int (optional)	Random seed. Defaults to 2023.
save_calib_msg	bool (optional)	If True, save calibration data dictionary as MSGpack file. Defaults to False.
msg_path	string (optional)	Path to save MSGpack file If not provided, it automatically generate the path with dataname and number of calibration data. Defaults to None.

7.4 Fuction: make_calib_man

From given images and manually written function that takes an image path as input, create the preprocessed numpy files and a txt file containing their paths.

Table 7-6. make_calib_man

Parameter	Type	Description
pre_ftn	Callable	Pre-processing function that takes an image path as input.
data_dir	string	Directory of data to be used for calibration.
save_dir	string	Directory to save the pre-processed numpy files and txt file which contains their paths.
save_name	string (optional)	Name for resulting files. Numpy files will be saved under {save_dir}/{save_name}_npz directory. Text file will be saved in {save_dir}/{save_name}.txt. If it is not provided, it is set to the basename of data_dir.
anno_json	string (optional)	Path to an annotation json file for COCO format. When provided, make_calib function randomly selects samples considering class balance. Defaults to None.
file_format	string (optional)	Filename format using image_idx. Defaults to '%012d.jpg'.
max_size	int (optional)	Maximum size of the resulting calibration data. Defaults to -1, which means no limit on the number of the calibration data.
remove_npy	bool (optional)	If True, remove pre-existing numpy files. Defaults to False.
seed	int (optional)	Random seed. Defaults to 2023.
save_calib_msg	bool (optional)	If True, save calibration data dictionary as MSGpack file. Defaults to False.
msg_path	string (optional)	Path to save MSGpack file If not provided, it automatically generate the path with dataname and number of calibration data. Defaults to None.

Example codes for using these functions are provided in the [## Preparing Calibration Data](#) section.

7.5 Pre-processing Configurations

qubee supports the following pre-processing functions to make calibration data.

Table 7-7. Pre-processing function API

Pre-processing Type	Description
Datatype	The data type of the loaded data, with support for Image type.
GetImage	Get image tensor from image path using cv2 backend or image tensor. Note that this should be at the top of the list.
Pre-Order	The pre-processes listed below are executed in the order specified by the pre-order list, and they must be identical to the pre-order.
Pad	Pad image tensor.
Normalize	Normalize image tensor.
ResizeTorch	Resize the input image to the given size using torchvision.transforms.functional.resize
Resize	Resize image tensor to the given size using cv2.resize .
CenterCrop	Center crop the image tensor.
SetOrder	Set the order of axes of the given image tensor. Note that this should be at the very end.

You can write a yaml file as follows:

```
[Pre-processing Type]
  [Parameter]: [Argument]
  ...
```

```
# Example
Datatype: Image
GetImage:
  to_float32: false
  channel_order: RGB
Pre-Order: [ResizeTorch, CenterCrop, Normalize, SetOrder]
ResizeTorch:
  size: [256, 256]
  interpolation: bilinear
CenterCrop:
  size: [224, 224]
Normalize:
  mean: [0.485, 0.456, 0.406]
  std: [0.229, 0.224, 0.225]
  to_float_div255: true
SetOrder:
  shape: HWC
```

7.5.1 Pre-processing Parameters

Table 7-8. GetImage

Parameter	Type	Description
to_float32	bool (optional)	If True, set dtype as float32. Defaults to False.
channel_order	string (optional)	Channel order to load. Upper cases will be converted into lower cases. Defaults to "bgr".

Table 7-9. Pad

Parameter	Type	Description
shape	Tuple[int] (optional)	Expected padding shape (h, w). Defaults to None.
size_divisor	int (optional)	Pad images so that the the resulting image's width and height are divisible by size_divisor. Defaults to None.
pad_val	float (optional)	Values to be filled in padding areas when padding_mode is 'constant'. Defaults to 0.
right_bottom	bool (optional)	If True, it only pads to right and bottom. Defaults to False.

Table 7-10. Normalize

Parameter	Type	Description
mean	List[float] or np.ndarray	Normalization mean.
std	List[float] or np.ndarray	Normalization standard deviation.
to_float_div255	bool (optional)	Normalize image between [0, 255] into [0, 1] by dividing by 255 before normalizing with the mean and std. Defaults to False.

Table 7-11. ResizeTorch

Parameter	Type	Description
size	List[int]	Desired output size, i.e., height and width.
interpolation	string	Interpolation method, accepted values are "nearest", "bilinear", "bicubic", "box", "hamming", "lanczos".

Table 7-12. Resize

Parameter	Type	Description
img_scale	float or Tuple[int, int]	The scaling factor or maximum size (h, w). If it is a float number, then the image will be rescaled by this factor, else if it is a tuple of 2 integers, then the image will be rescaled as large as possible within the scale.
keep_ratio	bool	Whether to keep the aspect ratio when resizing the image. Defaults to False.
interpolation	string	Interpolation method, accepted values are "nearest", "bilinear", "bicubic", "area", "lanczos".

Table 7-13. CenterCrop

Parameter	Type	Description
size	List[int]	Desired output height and width.

Table 7-14. SetOrder

Parameter	Type	Description
shape	string	Desired data layout format, accepted values are "HWC", "CHW", "BHCW", "BCHW". Defaults to "HWC".

8. Open Source License Notice

PyTorch

- <https://github.com/pytorch/pytorch>
- BSD-like License

TensorFlow

- <https://github.com/tensorflow/tensorflow>
- Apache 2.0 License

ONNX

- <https://github.com/onnx/onnx>
- Apache 2.0 License

ONNX Runtime

- <https://github.com/microsoft/onnxruntime>
- MIT License

Keras

- <https://github.com/keras-team/keras>
- Apache 2.0 License

9. Copyright

Copyright© 2019-present, Mobilint, Inc. All rights reserved.

Confidential